try / except — Guide

pyspark-lakehouse-aws · background reference · updated 2026-08-07

What it's for

`try` / `except` exists to handle failure conditions you **expect can happen** and know how to respond to — a file might not exist, a network call might time out, a write to S3 might fail mid-job. It is not a general-purpose safety net for "something might go wrong somewhere in this code," and it is not a substitute for fixing bugs in your own logic.

The anatomy — all four clauses

```
try:
    risky_operation()
except SpecificError as e:
    handle_or_log(e)
else:
    only_runs_if_no_exception()
finally:
    always_runs_cleanup()
```

Clause	Runs when
`try`	Always — contains the code that might raise.
`except`	Only if a matching exception was raised in the `try` block.
`else`	Only if **no** exception was raised — lets you separate "code that might fail" from "code that should only run on success," so the `try` block stays as small as possible.
`finally`	Always, whether or not an exception occurred — for guaranteed cleanup (closing a file, releasing a lock).

Most code only needs `try` / `except`. `else` and `finally` are for specific situations, not defaults.

Standard rules

1 Catch specific exceptions, not bare `except:`

AVOID

```
try:
    df = spark.read.csv(path)
except:
    logger.error("read failed")
```

PREFER

```
try:
    df = spark.read.csv(path)
except AnalysisException:
    logger.exception("Bad path: %s",
path)
    raise
```

A bare `except:` catches everything — including `KeyboardInterrupt` and typos in your own code that raise `NameError` / `AttributeError`. It turns real bugs into silent, misleading failures.

2 Keep the `try` block scoped to one coherent unit of work

Not necessarily "one line" — but one logical phase. Wrapping unrelated operations together makes it unclear which one raised, and risks silently catching failures you never meant to guard against. (See the worked example below — this rule is what "acquire" vs "persist" as separate blocks is built on.)

3 Never silently swallow an exception

```
# Never do this — the failure vanishes with no trace
try:
    write_to_bronze(df)
except Exception:
    pass
```

Always do one of: log it with enough context to debug later, handle it meaningfully, or re-raise it.

4 Log with `logger.exception()`, then decide: re-raise or not

```
try:
    dimensions_df.write.mode("overwrite").csv(f"s3a://bronze/{file_name}/")
except Exception:
    logger.exception("Failed to write %s to bronze", file_name)
    raise
```

`logger.exception()` auto-attaches the full stack trace. The `raise` afterward matters: it logs useful context *and* still lets the failure propagate, so `spark-submit` exits non-zero and the run is visibly marked failed — it doesn't get treated as success just because the error was logged.

5 EAFP is the Pythonic default — this is not "avoid try/except everywhere"

```
# LBYL — Look Before You Leap
if key in config:
    value = config[key]
else:
    value = default

# EAFP — Easier to Ask Forgiveness than Permission (idiomatic Python)
try:
    value = config[key]
except KeyError:
    value = default
```

Python culture generally prefers EAFP over pre-checking conditions. Don't read "be careful with try/except" as "avoid it" — the goal is using it *deliberately*, not avoiding it reflexively.

6 In Spark specifically: "lazy evaluation" doesn't mean everything is lazy

It's tempting to assume that because Spark defers execution to an action, *any* problem with a read will only surface later, at the write. That's false for a whole category of failures — see the worked example below for how this was actually tested and proven wrong.

Worked example — how this was actually figured out

This section documents the real debugging/design sequence from this project, not a hypothetical.

> **1. Starting assumption**
>
> "Spark is lazy, so if the read is broken, I'll find out when I write — no need for a try/except around the read itself."

> **2. Tested it directly, with no action at all**
>
> ```
> from config.spark_session import get_spark_session
> from config.schemas import customers_schema
>
> spark = get_spark_session()
> df = spark.read.schema(customers_schema).csv("data/raw/does_not_exist.csv")
> print("got here without an error")
> ```
>
> Result — the exception fired immediately, before the `print` line ever ran:
>
> ```
> pyspark.errors.exceptions.captured.AnalysisException: [PATH_NOT_FOUND]
> Path does not exist: file:/opt/app/data/raw/does_not_exist.csv.
> ```
>
> **Conclusion:** the starting assumption was wrong. Path/file resolution happens *eagerly*, as part of building the DataFrame's logical plan — not deferred to an action. Spark's laziness applies to *executing* the plan (scanning real data, running tasks), not to *resolving* it.

3. Found the actual first action — and it was unguarded

```
dimensions_df = spark.read.schema(schema)...csv(...)   # eager: path resolved
here
row_count = dimensions_df.count()                       # <- this is the real
first ACTION
dimensions_df.write.mode("overwrite").csv(...)          # already had a
try/except
```

The row-count line is where *lazy*, content-level failures (a malformed row, a value that won't coerce to the declared schema type) would actually surface — not the write. It had no `try` / `except` around it at all, and this gap survived more than one round of edits before being caught, because attention kept going to the write block instead.

4. Checked the real exception hierarchy before writing any handler

Rather than guessing at import paths, this was verified directly against the project's own PySpark install:

```
>>> from pyspark.errors import AnalysisException, PySparkException
>>> AnalysisException.__mro__
(AnalysisException, PySparkException, Exception, BaseException, object)
```

Two takeaways: the correct import is `from pyspark.errors import AnalysisException` (the clean public path — not `pyspark.errors.exceptions.captured.AnalysisException`, which is just what appears in a raw traceback). And `PySparkException` is the shared parent of *all* PySpark-specific exceptions — catching it is a deliberate middle ground between one narrow type and bare `except Exception`.

5. Compared the four `raise` flavours with real output, not just syntax

Four minimal scripts, same simulated failure, only the final `raise` changed:

Form	What the traceback actually shows
`raise` (bare)	One exception, unchanged — no chaining language at all.
`raise NewException(...)`	Two exceptions, joined by "*During handling of the above exception, another exception occurred*" — added automatically by Python, even without `from`.
`raise NewException(...) from e`	Two exceptions, joined by "*The above exception was the direct cause of the following exception*" — same info as above, stated as deliberate rather than incidental.
`raise NewException(...) from None`	Only **one** exception shown — the original is fully suppressed, not just de-emphasized. The most surprising one: it hides real debugging detail, not just noise.

6. First real handler — too narrow a scope, and a message that overclaimed the cause

```
try:
    dimensions_df = spark.read.schema(schema)...csv(...)
except Exception as e:
    logger.exception("Failed to read %s", file_name)
    raise RuntimeError(
        f"could not read data from source file as file is not available:
{file_name}"
    ) from e
```

Two problems here, caught on review: (a) `.count()` was still outside this block — same gap as step 3, now hiding behind a fix that looked complete; (b) the hardcoded message "*file is not available*" asserts one specific cause while catching the broad `Exception`, which could mean the message is flatly wrong for whatever actually failed.

> **7. Landed on: group by pipeline phase, not by individual Spark call**
>
> The resolution wasn't "three try/except blocks" (one per Spark call — overkill) or "one big block" (loses the read-vs-write signal). It was recognizing that `.count()` isn't a separate concern from the read — it's just the moment the read plan actually executes. Read + count together form one logical phase ("acquire the data"); write is a separate phase ("persist the data"). Once `.count()` joined the read's block, the guarded scope grew to include real execution, not just analysis — so `except PySparkException` was widened back to `except Exception`, matching the write block's reasoning for the same underlying cause.

Final result

```
logger.info("Reading %s", file_name)
try:
    dimensions_df = spark.read.schema(schema).option("multiline", "true").csv(
        f"data/raw/{file_name}.csv", header=True
    )
    row_count = dimensions_df.count()
    logger.info("Read %d rows for %s", row_count, file_name)
except Exception:
    logger.exception("Failed to read %s", file_name)
    raise

try:
    dimensions_df.write.mode("overwrite").csv(f"s3a://bronze/{file_name}/")
except Exception:
    logger.exception("Failed to write %s to bronze", file_name)
    raise

logger.info("Wrote %s to bronze", file_name)
```

Bare `raise` in both blocks — no custom exception wrapping, no asserted cause. `logger.exception()` captures full context for the logs; the original exception, untouched, is what actually propagates and crashes the job.

When to avoid it

1 When you'd just be masking a bug, not handling a real failure

```
# Bad: hides typos and logic errors as if they were expected failures
try:
    for file_name, schema in SCHEMAS.items():
        ingest_dimensions(file_name, schema)
except Exception:
    logger.error("something went wrong")
```

Wrapping the entire pipeline loop in one broad `except` means a genuine bug gets reported identically to an expected operational failure. You lose the ability to tell them apart.

2 When the "error" case is actually common — use `if` instead

Exceptions are for exceptional conditions. If something happens routinely, a plain conditional is clearer.

3 When you can't do anything useful with the exception

If you catch an exception only to immediately re-raise it with no added logging, context, or type conversion, remove the `try` / `except` — it's pure noise.

4 In a batch pipeline, when "continue anyway" would be worse than "fail loudly"

For this ELT job: if one table's write genuinely fails, catching it, logging a warning, and moving to the next table would leave bronze **partially loaded but silently reported as a success**. A visibly crashed job (non-zero exit code) is easier to trust than a "successful" run that quietly skipped a table — this is why both blocks in the final code re-raise rather than continue.

5 **When you don't know what exception type to expect**

If you can't name the specific exception a piece of code might raise, that's usually a sign you don't yet understand its failure modes well enough — verify empirically (as in step 2 above) rather than guessing.

Official documentation

- **Python tutorial — Errors and Exceptions** — docs.python.org/3/tutorial/errors.html
- **Python built-in exception hierarchy** — docs.python.org/3/library/exceptions.html
- **PySpark errors module** (`pyspark.errors`) — spark.apache.org/docs/latest/api/python/reference/pyspark.errors.html